

تقرير التحليل المعماري والفجوات التقنية لنظام حجوزات عقارات جمصة (Gamasa Properties): تقييم مسارات الأمان، التزامن، والبنية التحتية التوطئة والتحليل المعماري العام للنظام

يمثل انتقال الأنظمة البرمجية العقارية من بيئة التطوير المعزولة إلى بيئة الإنتاج الفعلي مرحلة هندسية حرجية تتطلب تطابقاً تاماً لا تشوبه شائبة بين التوقعات الوظيفية للواجهات الأمامية والقيود الهيكلية الصارمة لقواعد البيانات. يُظهر التحليل المعماري الشامل والمراجعة الدقيقة لمسارات بيانات الحجوزات، سواء مسارات القراءة أو الكتابة، في نظام "عقارات جمصة" (Gamasa Properties) انحرافات جوهرية عن معمارية "قاعدة البيانات أولاً" (Database-First) التي تم اعتمادها كركيزة أساسية في تصميم الإصدار الأول للمشروع. تتجلى هذه الفجوات التقنية في ثلاثة محاور معمارية رئيسية: أولاً، تجاوز سياسات الأمان وتخطي قواعد انتقالات الحالة (State Transitions)؛ ثانياً، القصور الواضح في البنية التحتية لتخزين المستندات المالية والوثائق الحساسة؛ وثالثاً، غياب ذرية المعاملات (Atomicity) والاعتماد على تسلسلات غير آمنة مما يفتح ثغرات واسعة لحالات السباق (Race Conditions).

إن استمرار الاعتماد على منطق التطبيق (Application Logic) والواجهات الأمامية في معالجة تزامن الحجوزات وإدارة حالات السجلات بدلاً من تفويض هذه المهام المعقدة لطبقة قاعدة البيانات (PostgreSQL) لا يعرض النظام فقط لخطر الحجوزات المزدوجة وفقدان سلامة البيانات، بل يخالف المبادئ الهندسية الصارمة التي تفرضها تقنيات الواجهة الخلفية كخدمة مثل (Supabase). علاوة على ذلك، فإن غياب البنية التحتية المؤمنة لإيصالات الدفع يتزامن مع تحولات تنظيمية وقانونية صارمة يشهدها السوق العقاري والسياحي المصري حالياً، مما يحتم معالجة هذه الفجوات فوراً وبشكل جذري لضمان موثوقية المنصة، حماية حقوق الأطراف المتعاقدة، والامتثال للقوانين والتشريعات المحلية المعمول بها. يسعى هذا التقرير إلى تشریح هذه الفجوات، بيان أثرها التقني والتجاري، ووضع خطة معالجة تنفيذية متكاملة تسد هذه الثغرات قبل الإطلاق الفعلي للمنصة.

البيئة التشغيلية والتنظيمية: توافق البنية التقنية مع معطيات السوق المصري

لا يمكن تقييم المتطلبات المعمارية لنظام حجز عقاري بمعزل عن البيئة التشغيلية والتشريعية التي سيعمل ضمنها. يشهد السوق المصري حالياً ثورة رقمية وتنظيمية تفرض قيوداً وتحديات غير مسبوقة على تصميم قواعد البيانات وهيكلية المعاملات المالية. تتمركز هذه التحديات في شقين رئيسيين: تطور آليات الدفع الرقمي، والتحديثات التشريعية لقطاع الإيجارات قصيرة المدى.

التحول نحو المدفوعات اللحظية (IPN) والاعتماد على الإيصالات الرقمية

شهدت البنية التحتية المالية في مصر طفرة هائلة مع إطلاق البنك المركزي المصري لشبكة المدفوعات اللحظية (IPN) وتطبيق (InstaPay) في عام 2022.² بفضل هذه الشبكة، أصبحت التحويلات المالية بين الحسابات البنكية والمحافظ الإلكترونية (مثل Vodafone Cash التي تستحوذ على الحصة الأكبر من السوق) تتم بشكل فوري وعلى مدار الساعة.⁴ هذا التحول جعل الإيصالات الرقمية (Digital Receipts) ولقطات الشاشة لإثبات التحويلات بمثابة المستند القانوني والمالي الفعلي لإتمام أي صفقة عقارية أو تأكيد حجز إيجاري.³

يفرض هذا الواقع قيوداً معمارية صارمة على نظام "عقارات جمصة". فعندما يقوم المستأجر بتحويل مبلغ مالي (يصل إلى 70,000 جنيه مصري للمعاملة الواحدة كحد أقصى عبر InstaPay)⁷، فإنه يتوقع استجابة نظامية فورية وأمنة تضمن حفظ حقه. إن فقدان إيصال الدفع بسبب خطأ في مسارات التخزين أو عدم وجود حاوية مؤمنة مخصصة (كما سيتم تفصيله في المحاور اللاحقة) لا يمثل مجرد خطأ تقني (Bug)، بل يترتب عليه نزاع مالي معقد وفقدان تام لثقة المستخدمين في المنصة العقارية.⁸ بالتالي، يجب أن تُعامل البنية التحتية لتخزين إيصالات الدفع في قاعدة البيانات بنفس مستوى الأهمية الذي تُعامل به الجداول المالية الأساسية.

التوافق مع التشريعات السياحية وقرارات بيوت العطلات (Holiday Homes)

يمتد السياق التشريعي ليفرض المزيد من الصرامة على هيكل البيانات. في عام 2025، أصدرت وزارة السياحة والآثار المصرية (MoTA) مجموعة من القرارات الحاسمة، أبرزها القرار رقم 209 والقرار رقم 801 لعام 2025، والتي تهدف إلى تنظيم وتأطير قطاع الإيجارات قصيرة المدى وبيوت العطلات.⁹ تلزم هذه القرارات جميع الأفراد والكيانات المؤسسية التي تدير عقارات سياحية (من استوديوهات و وحدات فندقية) بالحصول على تراخيص رسمية وتوفير سجلات بيانات شفافة يمكن مراجعتها ضريبياً وأمنياً.⁹

يستوجب هذا التطور التشريعي أن تكون بنية قاعدة البيانات في "عقارات جمصة" منيعة ضد التلاعب (Immutable) وقادرة على الاحتفاظ بسجل تاريخي دقيق لكافة الحجوزات والمدفوعات والإلغاءات. لا يجوز معمارياً مسح سجلات الحجز (DELETE) بحجة الإلغاء، بل يجب نقل حالة السجل ضمن "آلة حالة" (State Machine) محكمة تعكس الواقع التعاقدية وتوفر مسار تدقيق (Audit Trail) موثوق للامتثال لمتطلبات مصلحة الضرائب المصرية (ETA) والجهات السياحية الرقابية.⁹ هذا المطلب يتقاطع بشكل مباشر مع ضرورة سد فجوة انتقالات الحالة وصلاحيات التحديث التي سيتم مناقشتها تالياً.

المحور الأول: فجوة الصلاحيات وإدارة انتقالات حالة الحجز (State Transitions & RLS Bypass)

تم تصميم معمارية أمان النظام لتعمل وفق منهجية الثقة المعدومة (Zero-Trust)، حيث يتم فرض سياسات أمان مستوى الصف (Row Level Security - RLS) داخل محرك (PostgreSQL) لضمان فصل الصلاحيات بين أدوار المستخدمين المتعددة (مستأجرين، ملاك، وإداريين).¹⁰ غير أن تدقيق مسارات تدفق البيانات الخاصة بإلغاء الحجوزات ورفع إيصال الدفع كشف عن انحراف خطير عن المخطط المعماري المعتمد (V1).

تشخيص الخلل المعماري: التحديث المباشر وتجاوز آلة الحالة

يُظهر التنفيذ الحالي لمسار "حجوزاتي" أن الواجهة الأمامية (Frontend) تحاول تعديل حالة الحجز (Status) بشكل مباشر عبر إصدار أوامر تحديث برمجية صريحة (مثال: `supabase.from('bookings').update({ status: 'cancelled' })`). يصطدم هذا السلوك بجدار الحماية الممنع المتمثل في سياسات RLS، والتي حُدَّت مسبقاً لحصر صلاحيات التحديث في المالك (Landlord) فقط، مما يؤدي إلى فشل حتمي لكافة محاولات المستأجرين لاستكمال عمليات الإلغاء أو إرفاق إثباتات الدفع.

تتمكن الجذور الهندسية لهذه الفجوة في التعامل مع حقل "حالة الحجز" (Booking Status) كمجرد حقل نصي قابل للتعديل، في حين أنه يمثل المحرك الأساسي لـ "آلة الحالة" (State Machine) التي تدير دورة حياة الحجز بأكملها.¹¹ في الأنظمة العقارية والمالية المعقدة، يتطلب الانتقال من حالة إلى أخرى (State Transition) استيفاء شروط مسبقة (Pre-conditions) وينتج عنه تأثيرات لاحقة (Side-effects) حتمية.¹³

إذا ما تم اللجوء إلى "الحل السهل" المتمثل في منح المستأجرين صلاحية UPDATE شاملة على جدول الحجوزات لتجاوز المشكلة الحالية، فإن ذلك سيشكل ثغرة أمنية كارثية (Privilege Escalation)؛ حيث سيتمكن أي مستخدم يمتلك دراية برمجية بسيطة من تجاوز الواجهة الأمامية وإرسال طلب واجهة برمجية تطبيقات (API Request) خبيث يحول حالة حجزه مباشرة من pending إلى active أو completed دون المرور بسلسلة الموافقات الإدارية أو الدفع المالي الفعلي.¹⁴ هذا التجاوز المباشر لسياسات العمل يدمر تكامل البيانات والموثوقية المالية للمنصة.

هندسة مسارات الإلغاء وتطبيق آلة الحالة الذرية

لمعالجة هذه المعضلة المعمارية، يجب التخلص نهائياً من أي محاولات لتحديث حالة الحجز مباشرة عبر الواجهة الأمامية. عوضاً عن ذلك، يجب توجيه الواجهة لاستدعاء دوال إجراءات عن بُعد (RPCs) تعمل كبوابات تحكم (Gatekeepers) تنفذ منطق العمل بأمان داخل بيئة قاعدة البيانات وتحت مظلة أمنية محكمة. يتطلب هذا المسار إعادة تعريف مصفوفة الحالات وإدراج حالات جديدة تسد الفجوات الإدارية.

يجب دمج حالة جديدة في نوع البيانات المخصص للحجوزات تحت مسمى `cancellation_requested`، وتصميم دالة مخصصة باسم `request_booking_cancellation(p_booking_id uuid)` تعمل كآلة حالة مصغرة. تتلقى هذه الدالة طلب الإلغاء وتفحص الحالة الحالية للسجل وتتصرف بناءً عليها:

- في حال كان الحجز في مراحله الأولية (`pending` أو `requested`)، يتم تحويل السجل فوراً إلى `cancelled`، حيث لم تترتب أي التزامات مالية أو قانونية معقدة بين الأطراف.¹²
- أما إذا كان الحجز في حالة مؤكدة (`confirmed`) وتم تحويل الأموال، فلا يحق للمستأجر أو النظام إلغاء الحجز تلقائياً. بدلاً من ذلك، تقوم الدالة بنقل الحالة إلى `cancellation_requested`. يضع هذا الانتقال الحجز في سياق تنظيمي معلق، مما يستدعي تدخلاً بشرياً من قبل الإدارة لمراجعة سياسات الاسترداد المالي، والتحقق من إمكانية إرجاع الأموال عبر (InstaPay) أو الحوالات البنكية بعد خصم الرسوم المستحقة إن وجدت.⁸
- تضمن الدالة الحماية القصوى بمنع أي تغيير (وإصدار استثناء `RAISE EXCEPTION`) إذا كان الحجز قد انتقل بالفعل إلى حالات التشغيل الفعلي (`active` أو `completed`)، مما يمنع التلاعب بالسجلات التاريخية النشطة.¹⁷

يتزامن مع هذا المسار إنشاء دالة إدارية موازية `admin_resolve_booking_cancellation(p_booking_id uuid, p_decision text)` تتيح للإدارة حسم الطلبات المعلقة إما بالقبول (`approve` لينتقل إلى `cancelled`) أو بالرفض (`reject` ليعود إلى `confirmed`). هذا التصميم يضمن بقاء سجلات العمليات نظيفة ويفصل بين رغبة المستخدم النهائية وبين الموافقة الإدارية النافذة.

تأمين دوال (SECURITY DEFINER) ومكافحة اختطاف مسار البحث

لنتمكن الدوال (RPCs) المقترحة من تعديل بيانات جدول الحجوزات نيابة عن المستأجر (الذي لا يمتلك صلاحية UPDATE صريحة)، يجب تعريف هذه الدوال بامتيازات SECURITY DEFINER. يسمح هذا الامتياز للدالة بالتنفيذ باستخدام صلاحيات المنشئ (عادةً المدير أو postgres)، مما يتيح لها تجاوز سياسات RLS مؤقتاً لتنفيذ منطق آلة الحالة.

ومع ذلك، يحمل هذا الامتياز مخاطر أمنية شديدة إذا لم يُنفذ بحذر بالغ. التحدي الأمني الأبرز هنا هو هجوم "اختطاف مسار البحث" (Search Path Hijacking). في بيئة PostgreSQL، يحدد متغير `search_path` الترتيب الذي يبحث فيه المحرك عن الكائنات (مثل الجداول والدوال) غير المؤهلة ببادئة المخطط (Schema Prefix).¹ إذا لم يتم تقييد هذا المسار داخل دالة SECURITY DEFINER، يمكن لمهاجم خبيث إنشاء دوال أو جداول وهمية في مخطط عام يحمل نفس أسماء الكائنات النظامية، مما يخدع الدالة لتنفيذ شيفرة المهاجم بصلاحيات إدارية مرتفعة، مؤدياً إلى اختراق شامل للقاعدة.¹⁹

لتحصين النظام ضد هذا السيناريو، يجب تطبيق إجراء إلزامي في تعريف كافة الدوال التي تتعامل مع انتقالات الحالة ورفع الإيصالات: يجب إدراج التعليمات `SET search_path =` في ترويسة الدالة.¹ هذا التقييد الصارم يجبر محرك قاعدة البيانات على رفض أي كائن غير مؤهل، ويلزم المطورين بكتابة الأسماء المؤهلة بالكامل (مثل `public.bookings` بدلاً من `bookings`) داخل جسم الدالة. يغلق هذا الإجراء المنهجي ثغرة تصعيد الصلاحيات بالكامل ويضمن تنفيذاً نظيفاً وأماناً لمنطق الأعمال.¹⁸ إضافة إلى ذلك، لابد من التحقق البرمجي الصارم داخل جسم الدالة من مطابقة `auth.uid` مع `user_id` الخاص بسجل الحجز، لضمان عدم استغلال الدالة للعبث بحجوزات المستخدمين الآخرين.

المحور الثاني: فجوة البنية التحتية لتخزين المستندات ومسارات العرض الآمنة

في قطاع التكنولوجيا العقارية (PropTech)، تُعد إدارة أصول الوسائط (Media Assets) والمستندات بفاعلية وأمان أمراً لا يقل أهمية عن إدارة الجداول العلائقية. نص المخطط المعماري لنظام "عقارات جمصة" على تخصيص بنية تحتية للتخزين معزولة لإيصالات الدفع، نظراً لحساسية هذه الوثائق. ومع ذلك، أظهر التدقيق للمشروع فجوة خطيرة تتمثل في غياب حاوية التخزين `payment-receipts` من ملفات تهيئة قاعدة البيانات (Migrations)، رغم استدعائها المستمر داخل طبقة الخدمات البرمجية للواجهة.

الأثر التشغيلي لغياب حاوية الدفع (Storage Bucket)

يعتمد إكمال دورة الحجز المؤكد على تقديم المستأجر لإيصال مالي يثبت إتمام عملية الدفع (عبر InstaPay أو الحوالات البنكية). في الوضع الحالي، يحاول الكود توجيه الملف المرفوع إلى حاوية غير موجودة في قاعدة البيانات الفعلية. سيؤدي هذا فوراً إلى أخطاء استجابة وقت التشغيل (Runtime Error 500)، مما يعطل التدفق النقدي كلياً ويمنع المستخدمين من تأكيد حجوزاتهم.²¹ إن ترك هذه البنية التحتية ليتم إنشاؤها يدوياً عبر لوحة تحكم Supabase يكسر مبدأ "البنية التحتية ككود" (Infrastructure as Code) ويخلق تبايناً مميئاً بين بيئات التطوير والاختبار والإنتاج.²² لذلك، من الضروري دمج إنشاء الحاوية كجزء أصيل من ملفات الهجرة (SQL Migrations).

هندسة العزل وتأمين سياسات الوصول للحاويات (Storage RLS)

لا يكفي مجرد إنشاء الحاوية، بل يجب تصنيفها بشكل افتراضي كحاوية خاصة (Private Bucket) وتأمينها عبر سياسات وصول دقيقة. في Supabase Storage، تتكامل سياسات أمان مستوى الصف (RLS) مع محرك تخزين الكائنات لتوفير تحكم دقيق في من يمكنه رفع أو قراءة الملفات.²³

يُعد تطبيق مبدأ "عزل المستأجر" (Tenant Isolation) المعيار الذهبي في هذا السياق. يجب ألا يتسنى لأي مستخدم، تحت أي ظرف، استعراض إيصالات مالية تعود لمستخدمين آخرين.²⁵ يتم تحقيق ذلك هندسياً من خلال إجبار النظام على هيكلة مسارات تخزين الملفات بحيث يكون المجلد الجذري لكل ملف هو المعرف الفريد للمستخدم user_id. بالتالي، تُصاغ سياسة الأمان الخاصة بعمليات الإدراج (INSERT) والقراءة (SELECT) لتتطابق المسار الأول للملف مع رمز المستخدم المستخرج من المصادقة:

SQL

```
create policy "Tenant can upload and read their own receipts"
on storage.objects for all to authenticated
using (
    bucket_id = 'payment-receipts' and
    (storage.foldername(name)) = auth.uid()::text
);
```

هذه السياسة المدعومة بمحرك PostgreSQL تضمن حماية الأصول الرقمية على مستوى التخزين السحابي بغض النظر عن الواجهة التي يطلب منها المستخدم البيانات، مما يسد جميع الثغرات الأمنية المحتملة المتعلقة بالوصول غير المصرح به.²⁴

تأمين العرض عبر الروابط الموقعة (Signed URLs) وأداء التطبيقات التقديمية (PWA)

إلى جانب تأمين التخزين، يتطلب عرض الإيصالات للمسؤولين وملاك العقارات آلية أمانة لا تكشف المسارات الفعلية للملفات للعموم. يفرض التصميم المستهدف استبدال الروابط العامة (Public URLs) بالروابط الموقعة (Signed URLs). تقوم هذه الآلية بتوليد رابط مؤقت مشفر يحمل توقيعاً زمنياً محدوداً (مثلاً 60 ثانية) يتيح عرض الملف وينتهي مفعوله فوراً.²³

يتم تمرير مسار التخزين المجرد (Storage Path) من الواجهة الأمامية إلى الدالة الذرية attach_booking_payment_proof ليتم حفظه في قاعدة البيانات كمرجع نصي لا ك رابط عام.²⁹ عند الحاجة لعرض الإيصال في لوحة تحكم الإدارة، تستدعي الواجهة الأمامية طلب توليد رابط موقع للمسار المخزن، مما يضمن ألا تُعرض المستندات المالية إلا للمسؤولين الموثوقين وفي جلسات محمية.²⁴

وبالنظر إلى أن النظام مبني كتطبيق ويب تقدمي (PWA)، يجب مراعاة سيناريوهات انقطاع الاتصال أو بطء الشبكات الخلوية، وهي شائعة في بعض المناطق الساحلية المصرية. إن محاولة رفع صور عالية الدقة لإيصالات الدفع قد تفشل، لذا توصي الممارسات الهندسية

بتجهيز الواجهة الأمامية بتقنيات المزامنة في الخلفية (Background Sync) والتخزين المؤقت في (IndexedDB) لضمان حفظ الملف محلياً وإعادة محاولة رفعه تلقائياً عند استقرار الاتصال، مما يعزز مرونة تجربة المستخدم ويقلل من معدلات التخلي عن الحجز.³¹

المحور الثالث: فجوة التزامن وذرية المعاملات (Atomicity & Race Conditions)

إذا كانت فجوات الصلاحيات والبنية التحتية تهدد الاستقرار التشغيلي، فإن ثغرة التزامن (Concurrency) تمثل الخطر المعماري الأكبر الذي يهدد صلب الأعمال التجارية في قطاع التكنولوجيا العقارية (PropTech). لقد أظهر التقييم للمسار الحالي الخاص بإنشاء الحجوزات (createBooking في supabaseService.ts) تبنيه لنمط مضاد (Anti-pattern) كلاسيكي يُعرف بـ "اقرأ ثم اكتب" (Read-then-Write).³³

تشريح الخلل المنهجي في حالات السباق

في التنفيذ القائم، تقوم الواجهة الأمامية (أو طبقة الخدمة) بالاستعلام عن قاعدة البيانات لفحص ما إذا كان العقار متاحاً في التواريخ المطلوبة. إذا عادت النتيجة بالإيجاب، يتم تنفيذ أمر الإدخال insert. هذا النهج يفترض خطأً أن النظام يعمل في بيئة تسلسلية (Sequential) ومعزولة. لكن في واقع أنظمة الويب الحديثة المبنية على بنى تحتية موزعة وعديمة الحالة (Serverless) مثل بيئة عمل Supabase، يمكن للطلبات أن تتزامن في أجزاء من الألف من الثانية.³⁴

إذا افترضنا أن مستخدمين مختلفين حاولوا حجز "شاليه" في نفس اللحظة لذات التواريخ المزدحمة، فإن كلا الطلبين سيستعملان عن التوفر في نفس الوقت، وسيحصل كلاهما على استجابة تؤكد إتاحة العقار. بناءً على ذلك، سيقوم كلا الطلبين بتنفيذ أمر الإدخال بنجاح. النتيجة الكارثية هي حدوث "حجز مزدوج" (Double Booking) أو ما يُعرف بـ (Race Condition) في السوق العقاري، يترتب على الحجز المزدوج تبعات مالية وقانونية جسيمة، وفقدان تام للموثوقية لدى الملاك والمستأجرين على حد سواء.

التخلي عن أقفال التطبيق والاعتماد على متانة قاعدة البيانات

تُظهر الممارسات الهندسية أن محاولة حل حالات السباق عبر أقفال في الذاكرة (In-memory Mutexes) أو عبر مستوى التطبيق (Application-level locks) تبوء بالفشل حتماً عند التوسع الأفقي للخوازم ³³. (Horizontal Scaling) الحل الجذري والوحيد المعترف به لمشكلات التداخل الزمني يكمن في تفويض المسؤولية بالكامل لطبقة تخزين البيانات واستغلال إمكانيات محرك (PostgreSQL) العميقة لتوفير حائط صد هيكلي لا يمكن تجاوزه.³⁵

توفر قواعد بيانات PostgreSQL تقنية متقدمة جداً تُعرف بـ "قيود الاستبعاد الحصرية" ³⁵ (Exclusion Constraints) على عكس قيد التفرد (UNIQUE Constraint) الذي يقتصر على مقارنة قيمتين متطابقتين تماماً، يسمح قيد الاستبعاد بتطبيق شروط أكثر تعقيداً على العلاقات المكانية أو الزمنية، لبضمن أنه لا يوجد أي صفين في الجدول يحققان شرط التداخل معاً.³⁵

لتحقيق هذا المستوى من ذرية المعاملات وحسم تضارب التواريخ، يجب الاعتماد على الفهارس من نوع (GiST - Generalized Search Tree). صُممت فهرس GiST للتعامل بكفاءة خارقة مع البيانات المعقدة التي تحتوي على نطاقات (Ranges) أو بيانات هندسية، مما يتيح لمحرك القاعدة اكتشاف التداخلات الزمنية (Overlaps) بسرعة فائقة ودون الحاجة لفحص كامل السجلات.³⁷

هندسة قيد الاستبعاد (EXCLUDE USING gist) ومعمارية الفترات الزمنية

لتأصيل هذا الحل في نظام "عقارات جمصة"، يتوجب كتابة ملف هجرة SQL يطبق قيد الاستبعاد بذكاء ويدمج الأنواع العددية مع النطاقات الزمنية. يتطلب ذلك أولاً تفعيل امتداد btree_gist الداخلي.³⁵

القيد المستهدف يتم صياغته لفرض القاعدة التالية: "يُمنع تقنياً وهيكلياً في قاعدة البيانات تسجيل أي حجزين يشتركان في نفس العقار property_id، وتتداخل نطاقاتهما الزمنية الممتدة من تاريخ البدء إلى تاريخ الانتهاء، طالما أن هذه الحجوزات لا تزال سارية أو قيد

التنفيذ الدقيق لهذا القيد يتطلب صياغة مشابهة للآتي:

SQL

CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE bookings

ADD CONSTRAINT prevent_double_booking

EXCLUDE USING gist (

property_id WITH =,

daterange(start_date, end_date, '

2. **دلالة الفترات نصف المفتوحة ('[)' - Half-open intervals): هذا المعامل الدقيق يخبر محرك القاعدة بأن تاريخ البدء 'start_date' هو جزء مشمول من الحجز (مغلق '[')، بينما تاريخ الانتهاء 'end_date' غير مشمول فعلياً ضمن الاستحواذ الزمني (مفتوح ')'. [39] هذه الدلالة الرياضية بالغة الأهمية للسوق العقاري، لأنها تسمح للمستأجر (أ) بالمغادرة في صباح يوم 15 من الشهر، وتسمح للمستأجر (ب) بالدخول واستلام العقار في ظهيرة نفس اليوم (يوم 15) دون أن يعتبر النظام ذلك تداخلاً ومخالفة للقيد. [39, 40]

3. **القيد الجزئي ('WHERE'):** يضمن هذا الشرط أن يتم استثناء الحجوزات الملغاة من عملية الفحص، مما يسمح بحجز التواريخ التي تم تحريرها مسبقاً، ويحافظ في نفس الوقت على السجلات الملغاة لأغراض التدقيق التاريخي، متوافقاً مع متطلبات الامتثال للمنظومة السياحية والضرورية المصرية.

دالة إنشاء الحجز الذرية (create_atomic_booking) والتقاط الأخطاء

بمجرد تثبيت هذا القيد، يتم تجريد الواجهة الأمامية من مسؤولية فحص التوفر، ويصبح مسار إنشاء الحجز محصوراً في استدعاء الدالة الذرية 'create_atomic_booking'. تتسلم هذه الدالة التفاصيل وتبدأ معاملة (Transaction) تقوم بإدراج الحجز مباشرة في الجدول.

إذا التقط المحرك تداخلاً زمنياً، سيرفض المعاملة بالكامل وبشكل ذري، مصدراً خطأ فنياً برمز الحالة '23P01' والذي يشير صراحة إلى [35, 41]. [Exclusion Violation] يتمثل دور طبقة الخدمات في الواجهة في التقاط هذا الرمز السلوكي وتفسيره، وتحويله إلى رسالة خطأ واضحة في واجهة المستخدم ("عذراً، لقد تم حجز هذه التواريخ قبل لحظات. يرجى اختيار مواعيد بديلة")، مما يوفر تجربة مستخدم سلسة وشفافة حتى في أسوأ سيناريوهات الضغط العالي والتزامن. [35] هذه المعمارية تجعل الحجوزات المزدوجة مستحيلة رياضياً وهيكلية على مستوى التخزين. [36]

المحور الرابع: جداول المقارنات التفصيلية (As-is vs. Should-be)

يُخلص الجدول التالي، بوضوح هيكلي، التباين الشديد بين التنفيذ الحالي المعرض للأخطاء، والتنفيذ الهندسي المستهدف الذي يعالج جذور الفجوات المتعددة:

المكون / النظام الفرعي	التنفيذ الحالي المكتشف (As-is)	المعمارية المستهدفة للإنتاج (Should-be)	الأثر ومستوى الخطورة (Severity)
تحديث حالة الحجوزات وإلغاؤها	الواجهة تعتمد على إرسال 'update()' مباشر، فتصطدم برفض سياسات 'RLS' للمستأجرين.	توجيه الواجهة لاستدعاء دوال 'RPC' مثل 'request_cancellation'، لتدبير القاعدة آلة الحالة.	**PO** (حرج): تعطيل كامل لسير العمليات، فلا يستطيع العميل إرفاق الدفع أو طلب الإلغاء، مما يهدد السيولة المالية للمنصة.
إدارة البنية التحتية للمستندات	حاوية 'payment-receipts' غير مبرمجة في الهجرة، وموجودة كمرجع كودي فقط مع روابط عامة.	إنشاء الحاوية عبر SQL ضمن ملفات الهجرة، وتطبيق 'RLS' للعزل، واستخدام الروابط الموقعة.	**PO** (حرج): فشل تقني مؤكد Error 500

عند الدفع، وانكشاف وثائق التحويل البنكي وانتهاك خصوصية المستخدمين. |
|**التزامن وتضارب التواريخ**| نمط "اقرأ ثم اكتب" غير الآمن من وجهة التطبيق، مما يسمح بفجوات لحالات السياق (Race Conditions). |
استخدام قيد حصري 'EXCLUDE USING gist' مع فترات نصف مفتوحة، وإدراج ذري للمعاملات. |**PO (حرج):** احتمالية حتمية
للحجوزات المزدوجة، مما يسبب نزاعات مالية وقانونية مع الملاك وفقدان السمعة. |
|**مرحلة دورة حياة الإلغاء**| غياب التمييز الهيكلي. الحجز إما ملغى أو نشط بشكل قاطع عبر استعلام الواجهة. | استحداث حالة
'cancellation_requested' للحجوزات المؤكدة، تستدعي تدخل لوحة تحكم الإدارة. |**P1** (عالي):** تسرب إيرادات وفوضى تشغيلية لعدم
قدرة النظام على حجز الأموال لمراجعتها قبل إرجاعها (Refunds). |

المحور الخامس: خطة المعالجة التنفيذية والهندسية (Remediation Action Plan)

بناءً على التوصيات والتقييمات المستفيضة، تتطلب جاهزية النظام للإنتاج المضي قدماً في تنفيذ حزمة من التعديلات الجذرية، مرتبة وفق التسلسل الهندسي الأمثل، وتُصنف جميعها ضمن مهام الأولوية القصوى (P0).

الإجراء الأول: تأسيس وحماية بنية إيصالات الدفع (Storage Infrastructure)

تهدف هذه الخطوة إلى بناء ملاذ آمن وموثوق للمستندات المالية، بما يتوافق مع سياسات الخصوصية والرقابة التنظيمية للتعاملات المالية (InstaPay) وغيرها):

1. **بناء الحاوية عبر الهجرة:** صياغة ملف هجرة جديد (مثال: 'payment_receipts_setup.sql_008') لإدراج سجل الحاوية 'payment-receipts' في نظام 'storage.buckets'، وتكوينها كحاوية معزولة ('public: false') تحد من وصول العموم. [42]
2. **سياسات عزل المستأجر (Tenant RLS):**
* كتابة سياسة تسمح للمستأجرين الموثقين ('authenticated') برفع الملفات (INSERT) بشرط مطابقة الدليل الجذري لمسار الملف مع المعرف الشخصي للمستأجر ('storage.foldername(name)) = auth.uid()::text').
* كتابة سياسة تسمح لمديري النظام (أو ملاك العقار المعيّنين) بالوصول وعرض هذه الملفات بناءً على جداول صلاحياتهم المدمجة. [1, 26]
3. **دعم الروابط الموقعة:** تحديث مسار عرض الحجز في الواجهة الأمامية ليقيم بطلب توليد رابط موقع ('createSignedUrl') بمدة زمنية لا تتجاوز دقيقة واحدة لعرض الإيصالات ضمن لوحة الملاك أو حجوزاتي، وتجنب حفظ أو مشاركة روابط دائمة. [30, 43]

الإجراء الثاني: تطبيق الذرية وتثبيت قيود الاستبعاد (Atomicity & Concurrency)

يعتبر هذا الإجراء بمثابة خط الدفاع الأخير الذي لا يمكن اختراقه لمنع الحجوزات المزدوجة:

1. **صياغة وتطبيق قيد GiST:** ضمن ملف هجرة جديد (مثال: 'atomic_concurrency_fix.sql_009')، يجب إضافة التوجيه 'CREATE EXTENSION IF NOT EXISTS btree_gist'، يليه تطبيق القيد 'EXCLUDE USING gist' على جدول 'bookings' لضمان عدم تقاطع 'daterange' الخاص بـ 'start_date' و 'end_date' لنفس 'property_id' للفترات نصف المفتوحة. [35]
2. **كتابة دالة 'create_atomic_booking':** تطوير الدالة بلغة (PL/pgSQL) لاستلام معلمات الحجز ومحاولة إدراجها.
3. **تحديث طبقة خدمة Supabase:** ('supabaseService.ts') إزالة التسلسل القديم القائم على فحص التوفر بالاستعلام العادي. بدلاً منه، يجب مناداة الدالة الذرية باستخدام 'supabase.rpc('create_atomic_booking', {...})'.
4. **معالجة رمز الاستبعاد في الواجهة:** بناء آلية مخصصة في الكود لالتقاط رمز الخطأ '23P01' وتوجيه رسالة ودية للمستخدم تخبره بأن العقار قد حُجز للتو من قبل مستخدم آخر، مما يضمن استمرارية التجربة البرمجية دون أعطال تقنية معقدة. [35]

الإجراء الثالث: هندسة آلة الحالة وتوحيد مسارات التحديث (State Transitions & RPCs)

يجب سلب صلاحية التحديث المباشرة وإسناد المهام الحرجة إلى دوال أمانة (SECURITY DEFINER) لمنع تصعيد الصلاحيات، مع إضافة وتحديث حالات الإلغاء:

1. **توسيع نطاق الحالات (Enums):** تعديل نوع البيانات المخصص لحالة الحجز في قاعدة البيانات ليضم الحالة الجديدة 'cancellation_requested'، وتحديث التعريفات المقابلة في 'database.types.ts' لتجنب أخطاء النوع في بيئة TypeScript.

2. **تصميم دالة `request\_booking\_cancellation`:

 - * إنشاء الدالة وتحديد صلاحياتها كـ `SECURITY DEFINER`.
 - * إلزام الدالة بمسار بحث فارغ `SET search\_path = ""` لتحديد مخاطر الاختطاف الموضحة سابقاً.
 - * تضمين الفحص البرمجي للأمان: `IF auth.uid() != booking\_owner\_id THEN RAISE EXCEPTION 'Unauthorized'`.
 - * تطبيق منطق آلة الحالة: النقل إلى `cancelled` أو `cancellation\_requested` بناءً على وصول المدفوعات من عدمه.

3. **تصميم دالة إرفاق الإثبات `attach\_booking\_payment\_proof`:

 - * دالة `RPC` محصنة تقوم فقط بتحديث حقل `payment\_receipt\_url` بمسار التخزين المجرّد (مثال: `payment-receipts/uid/receipt-123.jpg`) وتغيير الحالة إن استدعى الأمر.

4. **بناء واجهات التحكم الإدارية:

 - * إضافة صفحات مسار ولوحة تحكم مخصصة للمديرين (`admin/bookings/`) تعرض السجلات المعلقة، مصحوبة بدالة الـ `RPC` `admin\_resolve\_booking\_cancellation` لتأكيد الإلغاء المالي أو رفضه وإعادة حالة التأكيد، مع توضيح حالة الانتظار بشكل ملحوظ (Badges) للمستأجر في صفحة `حجوزاتي`.

المحور السادس: خطة الاختبار الشاملة وضمان الجودة (Comprehensive Test Plan)

لا تكتمل هذه التحديثات المعمارية دون خطة اختبار صارمة تضمن صلابة الحلول الهندسية وتوافقها التام مع متطلبات بيئة الإنتاج:

1. **اختبارات منع حالات السياق (Concurrency Tests):** محاكاة طلبات حجز متزامنة (برمجياً عبر سكريبتات إجهاد) تستهدف نفس العقار في نفس التواريخ المتقاطعة جزئياً وكلياً. يُعتبر الاختبار ناجحاً حصراً إذا تم قبول معاملة واحدة ورفض الباقي بقيمة خطأ `23P01` من قاعدة البيانات. [44, 35]
2. **اختبارات دلالة نصف المفتوح (Boundary Testing):** حجز عقار من يوم 10 إلى يوم 15. محاولة حجز نفس العقار من يوم 15 إلى يوم 20. يجب أن تنجح العملية ولا يعتبرها النظام تداخلاً بفضل صياغة القيد `[]` [39].
3. **اختبارات عزل التخزين (Storage Isolation):** دخول مستأجر بحساب "أ"، ومحاولة استرجاع وقراءة إيصال دفع يخص المستأجر "ب" عبر استدعاءات الواجهة المبرمجة وتعديل المسار يدوياً. يجب أن تفشل العملية قطعياً بنتيجة `Unauthorized 403` أو مسار فارغ بفضل تفعيل RLS على الحاوية الخاصة. [45, 24]
4. **اختبارات اختطاف مسار البحث (Search Path Security):** محاولة إنشاء دالة أو كائن يحمل اسماً مشابهاً لمكونات النظام في مخطط عام (Public Schema)، ثم استدعاء دوال الـ RPC الخاصة بالحالة، للتأكد من أن الدالة تعتمد حصرياً على الكائنات المؤهلة وأنها مُحصنة بـ `SET search\_path = ""` [1, 19].
5. **اختبارات آلة الحالة (State Machine Integrity):** محاولة إلغاء حجز في حالة `active` أو `completed` من قبل المستأجر. يجب أن يُقابل الطلب بالرفض المطلق (Exception)، بينما يسمح النظام بطلب الإلغاء للحجوزات `confirmed`، ناقلاً إياها لحالة الانتظار الإداري `cancellation\_requested` كما هو محدد. [47, 46]

الخلاصة والتوصيات النهائية

يُبرهن هذا التقييم المعماري المنهجي على أن الاعتماد الحصري على الواجهات الأمامية والمنطق التطبيقي (Application Logic) في إدارة أنظمة الحجوزات العقارية يشكل نقطة ضعف بالغة الخطورة، وخاصة في بيئات الأنظمة الموزعة والتخزين السحابي (Serverless Architectures). إن انتقال نظام "عقارات جمصة" نحو معمارية صارمة تعتمد "قاعدة البيانات أولاً" (DB-First) يمثل ضرورة تقنية، تجارية، وتنظيمية لا غنى عنها لتحقيق الاستقرار.

من خلال التحول الاستراتيجي إلى الإدارة المركزية لحالة الحجوزات (State Transitions) عبر دوال أمانة وذرية (Atomic RPCs) مدعومة بمسارات بحث مغلقة، وتطبيق قيود الاستبعاد الهيكلية (GiST Exclusion Constraints) لحل حالات التزامن بشكل قطعي ورياضي لا يقبل الخطأ، بالإضافة إلى بناء بيئة تخزين مستقلة ومشفرة لإيصالات الدفع (Payment Receipts) مدعومة بسياسات وصول متقدمة؛ يُرسى النظام أساساً خرسانياً ضد الاختراقات الأمنية وفوضى تضارب البيانات.

هذه التحسينات الهيكلية لا تصون فقط تجربة المستخدم وتمنع الخسائر التشغيلية، بل تُحاذي المنصة تقنياً وتجارياً مع الاشتراطات التشريعية الصارمة للتحول الرقمي المالي ومعايير السوق العقاري والسياحي المصري، ما يضع "عقارات جمصة" في موقع الريادة والموثوقية المطلقة عند تشغيل النظام في بيئة الإنتاج الفعلية.

Works cited

1. Supabase RLS Best Practices: Production Patterns for Secure Multi-Tenant Apps - MakerKit, accessed April 4, 2026, <https://makerkit.dev/blog/tutorials/supabase-rls-best-practices>
2. IPN Egypt - AfricaNenda, accessed April 4, 2026, https://www.africanenda.org/uploads/files/siips2025/siips_2025_IPN-Egypt_CaseStudy_en.pdf
3. Inside Egypt's InstaPay Economy: How Instant Payments Are Changing Access for a New Generation | Egyptian Streets, accessed April 4, 2026, <https://egyptianstreets.com/2026/01/04/inside-egypts-instapay-economy-how-instant-payments-are-changing-access-for-a-new-generation/>
4. Egypt's Payments Transformation: A Regional Hub in the Making - Thunes, accessed April 4, 2026, <https://www.thunes.com/insights/trends/egypts-payments-transformation-a-regional-hub-in-the-making/>
5. Vodafone Cash Government Payments, accessed April 4, 2026, <https://web.vodafone.com.eg/en/vodafone-cash-government-payments>
6. Electronic Receipt in Egypt | Guide for Registration and Implementation 2025 - Mofawtar, accessed April 4, 2026, <https://mofawtar.com/en/electronic-receipt-in-egypt/>
7. Q&A - instapay, accessed April 4, 2026, https://www.instapay.eg/?page_id=348&lang=en
8. Return & Refund Policy - Horizon Real Estate, accessed April 4, 2026, <https://thehorizonrealestate.com/return-refund-policy-hurghada-real-estate-projects/>
9. New Holiday Homes Regulation in Egypt — A Structured Regime for Short Term Rentals (Ministry of Tourism Decrees No. 209/2025 and No. 801/2025) - Amereller, accessed April 4, 2026, <https://amereller.com/publication/new-holiday-homes-regulation-in-egypt-a-structured-regime-for-short-term-rentals-ministry-of-tourism-decrees-no-209-2025-and-no-801-2025/>
10. Authorization via Row Level Security | Supabase Features, accessed April 4, 2026, <https://supabase.com/features/row-level-security>
11. Implementing State Machines in PostgreSQL - Felix Geisendörfer, accessed April 4, 2026, <https://felixge.de/2017/07/27/implementing-state-machines-in-postgresql/>
12. State Machine Diagram | LLD | AlgoMaster.io, accessed April 4, 2026, <https://algomaster.io/learn/llld/state-machine-diagram>
13. Saga Pattern for Resilient Flight Booking Workflows - DZone, accessed April 4, 2026, <https://dzone.com/articles/saga-state-machine-flight-booking>
14. Row Level Security | Supabase Docs, accessed April 4, 2026, <https://supabase.com/docs/guides/database/postgres/row-level-security>
15. 3. Use cases - Booking 2.0 - Digital Container Shipping Association (DCSA), accessed April 4, 2026,

- <https://dcsa.org/standards/booking/documentation-booking-2/booking-2-use-cases>
16. Design a Booking State Workflow in PHP Laravel - cmsGalaxy, accessed April 4, 2026,
<https://www.cmsgalaxy.com/blog/design-a-booking-state-workflow-in-php-laravel/>
 17. State machine represented on DB, enforcement of state transition - Stack Overflow, accessed April 4, 2026,
<https://stackoverflow.com/questions/39392300/state-machine-represented-on-db-enforcement-of-state-transition>
 18. Performance and Security Advisors | Supabase Docs, accessed April 4, 2026,
https://supabase.com/docs/guides/database/database-advisors?queryGroups=lint&lint=0011_function_search_path_mutable
 19. Abusing SECURITY DEFINER functions in PostgreSQL, accessed April 4, 2026,
<https://www.cybertec-postgresql.com/en/abusing-security-definer-functions/>
 20. Database Functions | Supabase Docs, accessed April 4, 2026,
<https://supabase.com/docs/guides/database/functions>
 21. Creating Buckets | Supabase Docs, accessed April 4, 2026,
<https://supabase.com/docs/guides/storage/buckets/creating-buckets>
 22. Migrating storage buckets : r/Supabase - Reddit, accessed April 4, 2026,
https://www.reddit.com/r/Supabase/comments/17ztxsc/migrating_storage_buckets/
 23. Storage Buckets | Supabase Docs, accessed April 4, 2026,
<https://supabase.com/docs/guides/storage/buckets/fundamentals>
 24. Storage Access Control | Supabase Docs, accessed April 4, 2026,
<https://supabase.com/docs/guides/storage/security/access-control>
 25. Mastering Supabase RLS - "Row Level Security" as a Beginner - DEV Community, accessed April 4, 2026,
<https://dev.to/asheeshh/mastering-supabase-rls-row-level-security-as-a-beginner-5175>
 26. Enforcing Row Level Security in Supabase: A Deep Dive into LockIn's Multi-Tenant Architecture - DEV Community, accessed April 4, 2026,
<https://dev.to/blackie360/-enforcing-row-level-security-in-supabase-a-deep-dive-into-lockins-multi-tenant-architecture-4hd2>
 27. Supabase RLS Guide: Policies That Actually Work - DesignRevision, accessed April 4, 2026, <https://designrevision.com/blog/supabase-row-level-security>
 28. Signed URL file uploads with NextJs and Supabase | by Ollie - Medium, accessed April 4, 2026,
<https://medium.com/@olliedoesdev/signed-url-file-uploads-with-nextjs-and-supabase-74ba91b65fe0>
 29. Best way to handle Supabase signed URLs for private images on dashboard?, accessed April 4, 2026,
<https://stackoverflow.com/questions/79588637/best-way-to-handle-supabase-signed-urls-for-private-images-on-dashboard>
 30. Bulk Request Signed URLs (instead of requesting individually for each URL) ·

- supabase · Discussion #27994 - GitHub, accessed April 4, 2026,
<https://github.com/orgs/supabase/discussions/27994>
31. Resumable uploads | Supabase Features, accessed April 4, 2026,
<https://supabase.com/features/resumable-uploads>
 32. Building a PWA with Supabase - Reddit, accessed April 4, 2026,
https://www.reddit.com/r/PWA/comments/16h1vmp/building_a_pwa_with_supabase/
 33. Race Conditions in Hotel Booking Systems: Why Your Technology Choice Matters More Than You Think - Amitav Roy, accessed April 4, 2026,
<https://www.amitavroy.com/articles/race-conditions-in-hotel-booking-systems-why-your-technology-choice-matters-more-than-you-think>
 34. Data Integrity First: Mastering Transactions in Supabase SQL for Reliable Applications, accessed April 4, 2026,
<https://dev.to/damasosanoja/data-integrity-first-mastering-transactions-in-supabase-sql-for-reliable-applications-2dbb>
 35. PostgreSQL's GiST Exclusion Constraint: The Database-Level ..., accessed April 4, 2026,
<https://amitavroy.com/articles/postgresql-gist-exclusion-constraintthe-database-level-answer-to-double-bookings>
 36. PostgreSQL's GiST Exclusion Constraint: The Database-Level Answer to Double Bookings, accessed April 4, 2026,
<https://www.amitavroy.com/articles/postgresql-gist-exclusion-constraintthe-database-level-answer-to-double-bookings>
 37. Implementing Mutual Exclusion Constraints with Exclude · Vonng - pigsty, accessed April 4, 2026, <https://blog.vonng.com/en/pg/sql-exclude/>
 38. Exclusion Constraints in Postgres | by Java Jedi - Medium, accessed April 4, 2026,
<https://java-jedi.medium.com/exclusion-constraints-b2cbd62b637a>